

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

КАТЕДРА ОБЧИСЛЮВАЛЬНОЇ ТЕХНІКИ

Алгоритми та методи обчислень

Лабораторна робота №1

«Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем»

Виконав:
студент групи ІО-32
Крадожон М.
Перевірів(-ла):
Порєв В.

Київ – 2023

Лабораторна робота №1

Тема: «Поняття алгоритму. Задавання алгоритмів у вигляді блок-схем»

Мета: Навчитися створювати блок-схеми лінійного алгоритму; розгалуженого алгоритму та циклічного алгоритму за допомогою редактора блок-схем аfсе або іншого довільного редактора.

Завдання: Відповідно до варіанту завдання розробити блок-схеми обчислення виразів для лінійного алгоритму, алгоритму, що розгалужується та циклічного алгоритму. У відповідності до блок-схеми створити програму обчислення виразу алгоритмічною мовою, узгодженою з викладачем.

Варіант: 15

15	$Y1 = \left(a + \frac{b}{x}\right)^3 + \left(b + \frac{a}{x}\right)^5$	Якщо $i = 2n + 1$ то $y = 25 \cdot c_i^2 - \sqrt{\frac{2 \cdot i}{34 \cdot x^2}} + 4 \cdot \sqrt{\frac{i}{45x^2}}$ інакше якщо $i = 2n$, то $y = 25 \cdot x^2 - \sqrt{\frac{2 \cdot i}{34 \cdot c_i^2}} + 4 \cdot \sqrt{\frac{i}{45c_i^2}}$	Обчислити $f = \prod_{i=1}^n (a_i^3 - b_i^3) + \sum_{i=1}^n (a_i^3 + b_i^3)$
----	--	---	---

Блок-схеми алгоритмів:

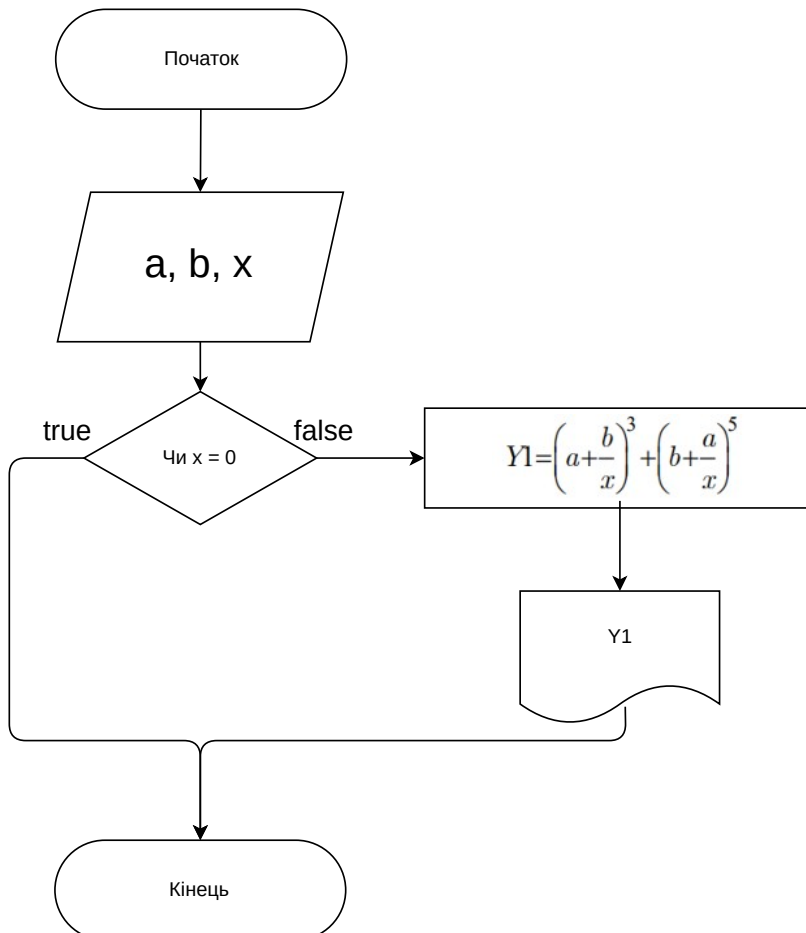
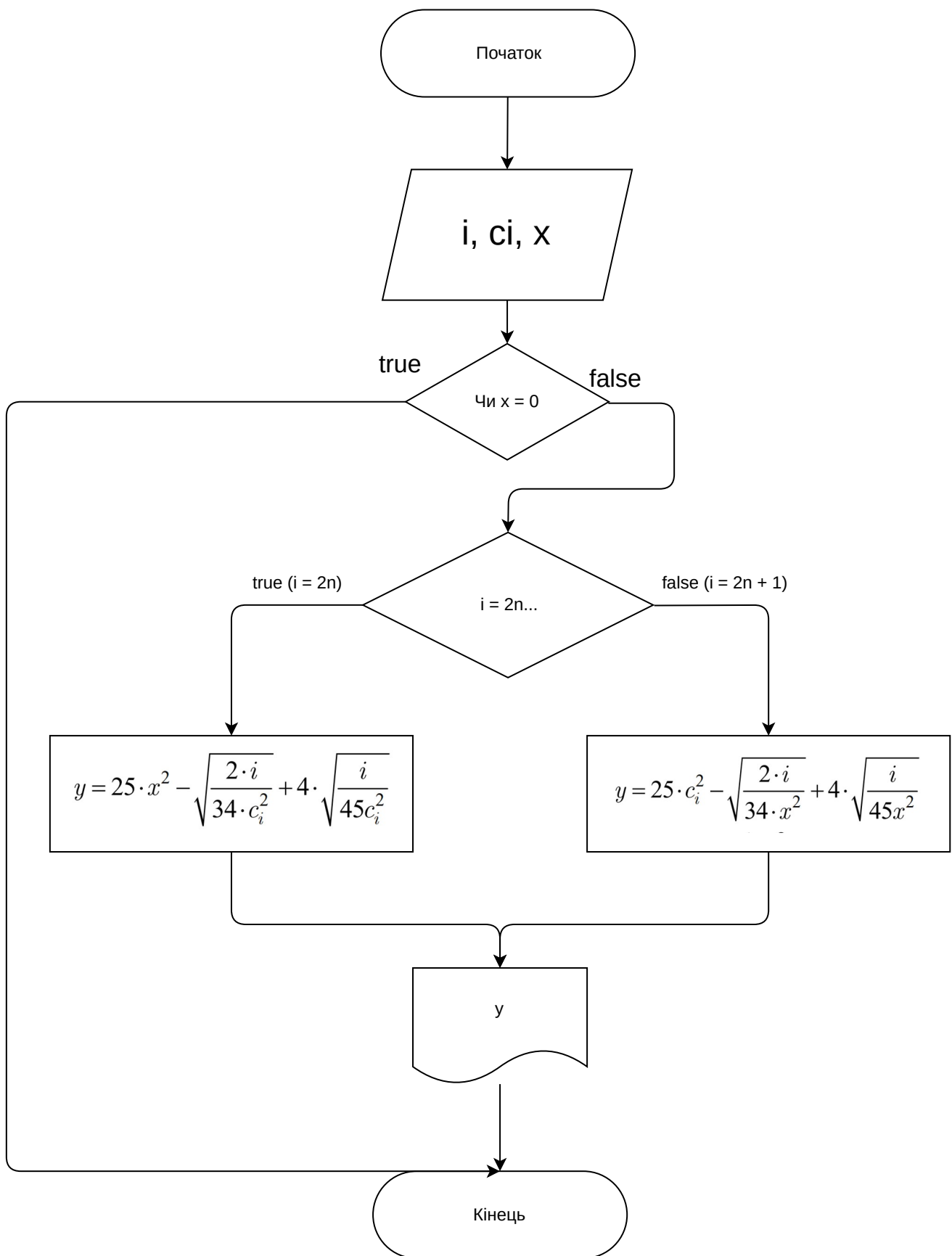


рис. 1. Лінійний алгоритм



%3CmxGraphModel

рис 2. Алгоритм, що розгалужується

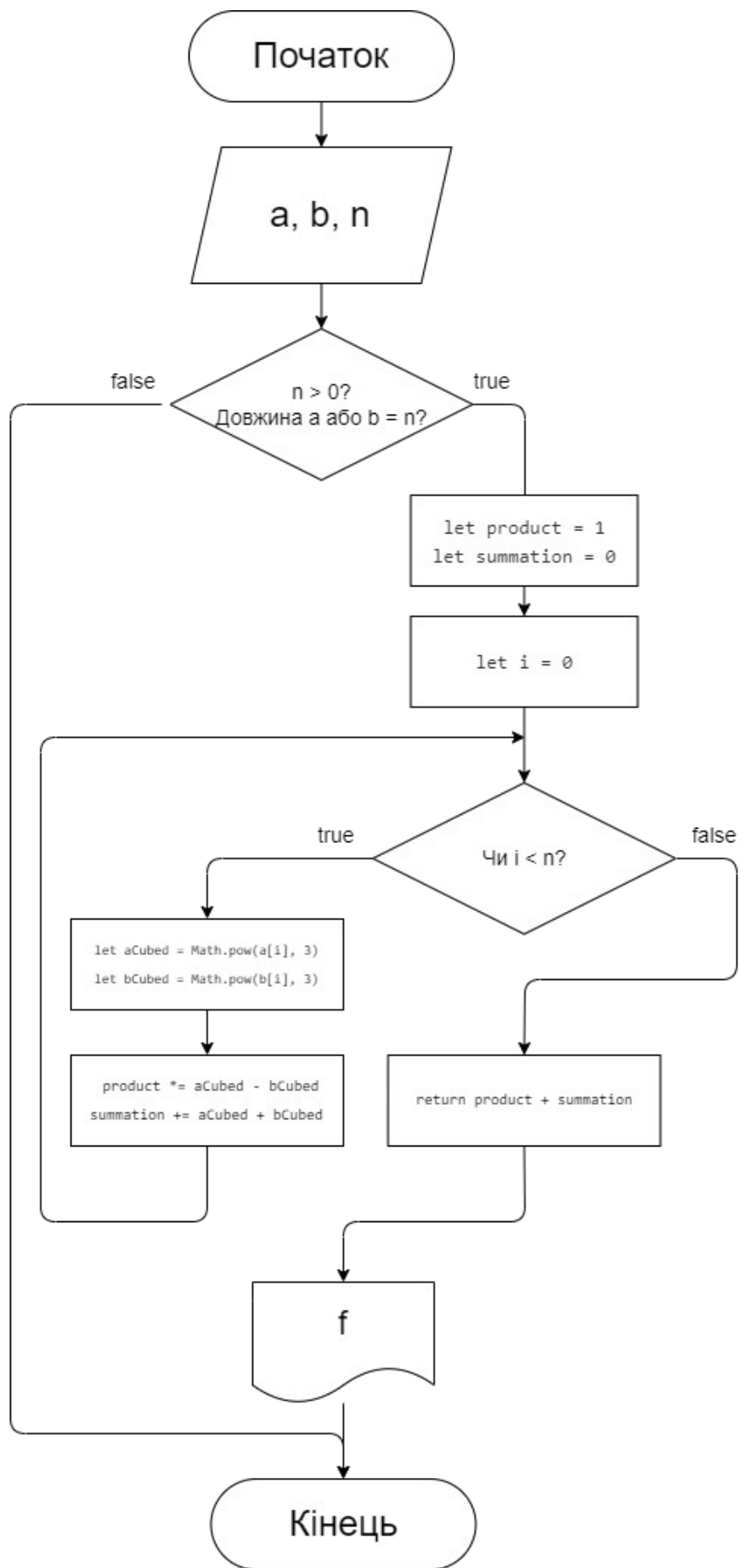


рис 3. Циклічний алгоритм

Код програми:

Алгоритми

```
const computeY1 = ({
  a,
  b,
  x
}: {
  a: number
  b: number
  x: number
}):
| number
| {
  type: 'info' | 'error'
  title: string
  message: string
} => {
  if (x === 0) {
    return {
      type: 'error',
      title: 'Помилка',
      message: 'x не може дорівнювати 0'
    }
  }
  return Math.pow(a + b / x, 3) + Math.pow(b + a / x, 5)
}

console.log(computeY1({ a: 1, b: 2, x: 1 }))

export { computeY1 }

const computeY = ({ i, ci, x }: { i: number; ci: number; x: number }) => {
  if (x === 0) {
    return {
      type: 'error',
      title: 'Помилка',
      message: 'x не може дорівнювати 0'
    }
  }
  if (i % 2 === 1) {
    // i = 2n + 1 (непарне)
    return (
      25 * ci ** 2 -
      Math.sqrt((2 * i) / (34 * x ** 2)) +
      4 * Math.sqrt(i / (45 * x ** 2))
    )
  } else {
    // i = 2n (парне)
    return (
      25 * x ** 2 -
```

```

    Math.sqrt((2 * i) / (34 * ci ** 2)) +
    4 * Math.sqrt(i / (45 * ci ** 2))
  )
}
}

```

```
export default computeY
```

```

const calculateF = ({
  n,
  a,
  b
}): {
  n: number
  a: number[]
  b: number[]
}):
  | number
  | {
    type: 'info' | 'error'
    title: string
    message: string
  } => {
  if (n <= 0) {
    return {
      type: 'error',
      title: 'Помилка',
      message: 'n має бути більше 0'
    }
  }
  if (a.length !== n || b.length !== n) {
    return {
      type: 'error',
      title: 'Помилка',
      message: 'Довжина масивів a та b має бути рівною n'
    }
  }

  let product = 1
  let summation = 0

  for (let i = 0; i < n; i++) {
    const aCubed = Math.pow(a[i], 3)
    const bCubed = Math.pow(b[i], 3)
    product *= aCubed - bCubed
    summation += aCubed + bCubed
  }

  return product + summation
}

```

```
export default calculateF
```

Програма та графічний інтерфейс

main.tsx

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import { createBrowserRouter, RouterProvider } from 'react-router-dom'

import OutletComponent from './components/outlet.tsx'
import BranchingPage from './pages/branching.tsx'
import LinearPage from './pages/linear.tsx'
import LoopingPage from './pages/looping.tsx'

import './index.css'

const router = createBrowserRouter([
  {
    path: '/',
    element: <OutletComponent />,
    children: [
      {
        path: 'linear',
        element: <LinearPage />
      },
      {
        path: 'branching',
        element: <BranchingPage />
      },
      {
        path: 'looping',
        element: <LoopingPage />
      }
    ]
  }
])

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <RouterProvider router={router} />
  </React.StrictMode>
)

// Use contextBridge
window.ipcRenderer.on('main-process-message', (_event, message) => {
  console.log(message)
})
```

outlet.tsx

```
import { Outlet } from 'react-router'

import Sidebar from './sidebar/sidebar'

function OutletComponent() {
  return (
    <div className="flex h-screen">
      <Sidebar />
      <Outlet />
    </div>
  )
}

export default OutletComponent
```

sidebar.tsx

```
import PathButton from './path-button'

function Sidebar() {
  return (
    <aside
      className="flex h-screen w-1/4 flex-col justify-between bg-gray-200 p-4 dark:border-r
dark:border-stone-800 dark:bg-neutral-950"
      aria-label="Sidebar Navigation"
    >
      <div>
        <h1 className="text-center text-xl font-bold dark:text-gray-200">
          Алгоритми
        </h1>

        <nav
          aria-label="Algorithm Categories"
          className="mt-4 flex flex-col gap-y-3"
        >
          <PathButton path="linear">Лінійні</PathButton>
          <PathButton path="branching">Розгалужувальні</PathButton>
          <PathButton path="looping">Циклічні</PathButton>
        </nav>
      </div>
      <div className="border-t pt-1 text-gray-300 dark:text-stone-800">
        <p className="py-2 text-sm text-gray-400 dark:text-neutral-500">
          Крадожон Максим Романович
        </p>
        <p className="text-sm text-gray-400 dark:text-neutral-500">
          IO-32 • 15 варіант
        </p>
      </div>
    </aside>
  )
}

export default Sidebar
```

path-button.tsx

```
import { PropsWithChildren, ReactNode } from 'react'
import { Link, useLocation } from 'react-router'
import clsx from 'clsx'

interface PathButtonProps {
  path: string
  children: ReactNode | string
}

function PathButton({ path, children }: PropsWithChildren<PathButtonProps>) {
  const location = useLocation()

  return (
    <Link to={path}>
      <button
        className={clsx(
          'w-full rounded-md border border-gray-300 bg-gray-50 p-2 transition-colors
          duration-200 ease-in-out hover:bg-gray-300 dark:border-stone-700 dark:bg-neutral-950
          dark:text-neutral-400 dark:hover:bg-stone-900',
          location.pathname.slice(1) === path && 'bg-gray-300 dark:bg-stone-800'
        )}
      >
        {children}
      </button>
    </Link>
  )
}

export default PathButton
```

linear.tsx

```
import GenericPage, { Input } from '@components/main-page/main-template'
import { computeY1 } from '@lib/linear-compute'

function LinearPage() {
  const inputs: Input[] = [
    {
      name: 'a',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення А:',
      required: true,
      valueAsNumber: true
    },
    {
      name: 'b',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення В:',
      required: true,
      valueAsNumber: true
    }
  ]
}
```

```

    },
    {
      name: 'x',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення X:',
      required: true,
      valueAsNumber: true
    }
  ]

  return (
    <GenericPage
      title="Лінійний алгоритм"
      imageSrc="../../../public/linear-example.jpeg"
      inputs={inputs}
      computeFunction={computeY1}
      filePickerAccept=".json"
    />
  )
}

```

```
export default LinearPage
```

branching.tsx

```

import GenericPage, { Input } from '@components/main-page/main-template'
import computeY from '@lib/branching-compute'

```

```

function BranchingPage() {
  const inputs: Input[] = [
    {
      name: 'i',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення I:',
      required: true,
      valueAsNumber: true
    },
    {
      name: 'ci',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення CI:',
      required: true,
      valueAsNumber: true
    },
    {
      name: 'x',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення X:',
      required: true,

```

```

    valueAsNumber: true
  }
]

return (
  <GenericPage
    title="Розгалужувальний алгоритм"
    imageSrc="../../../public/branching-example.jpeg"
    inputs={inputs}
    computeFunction={computeY}
    filePickerAccept=".json"
  />
)
}

export default BranchingPage

```

looping.tsx

```

import GenericPage, { Input } from '@components/main-page/main-template'
import calculateF from '@lib/looping-compute'

interface InputProps {
  a: number[]
  b: number[]
  n: number
}

function LoopingPage() {
  const inputs: Input[] = [
    {
      name: 'n',
      type: 'number',
      placeholder: 'Число',
      labelName: 'Значення N:',
      required: true,
      valueAsNumber: true
    },
    {
      name: 'a',
      type: 'string',
      placeholder: 'Масив чисел',
      labelName: 'Значення A:',
      required: true
    },
    {
      name: 'b',
      type: 'string',
      placeholder: 'Масив чисел',
      labelName: 'Значення B:',
      required: true
    }
  ]
}

```

```

const computeFunction = (data: InputProps) => {
  data.a = data.a.split(',').map(Number)
  data.b = data.b.split(',').map(Number)
  // implement the looping compute function
  return calculateF(data)
}

return (
  <GenericPage
    title="Циклічний алгоритм"
    imageSrc="../../../public/looping-example.jpeg"
    inputs={inputs}
    computeFunction={computeFunction}
    filePickerAccept=".json"
  />
)
}

export default LoopingPage

```

main-template.tsx

```

import { useEffect, useRef } from 'react'
import { SubmitHandler, useForm } from 'react-hook-form'
import { useFilePicker } from 'use-file-picker'

import InputField from '@components/input-field'
import isValidJson from '@utils/is-json'
import showDialog from '@utils/show-dialog'

interface Input {
  name: string
  type: 'string' | 'number'
  placeholder: string
  labelName: string
  required: boolean
  valueAsNumber?: boolean
}

interface Props {
  title: string
  imageSrc: string
  inputs: Input[]
  computeFunction: (
    data: any
  ) => number | { type: 'info' | 'error'; title: string; message: string }
  filePickerAccept: string
}

function GenericPage({
  title,
  imageSrc,

```

```

inputs,
computeFunction,
filePickerAccept
}: Props) {
  const { openFilePicker, filesContent } = useFilePicker({
    accept: filePickerAccept,
    multiple: false,
    readAs: 'Text'
  })
  const { register, handleSubmit } = useForm()
  const submitRef = useRef<HTMLInputElement>(null)

  const onSubmit: SubmitHandler<any> = (data) => {
    const computedValue = computeFunction(data)
    if (typeof computedValue === 'object') {
      showDialog(computedValue.type, computedValue.title, computedValue.message)
      return
    }
    showDialog('info', 'Результат', `y = ${computedValue}`)
  }

  useEffect(() => {
    if (!filesContent.length) return
    const content = filesContent[0].content

    if (!isValidJson(content)) {
      showDialog('error', 'Помилка', 'Файл містить некоректні дані')
      return
    }

    const jsonData = JSON.parse(content)
    const computedValue = computeFunction(jsonData)
    showDialog('info', 'Результат', `y = ${computedValue}`)
  }, [filesContent, computeFunction])

  return (
    <main className="flex flex-1 flex-col gap-4 p-4 dark:bg-black">
      <h1 className="text-center text-xl font-bold dark:text-gray-200">
        {title}
      </h1>
      <img
        src={imageSrc}
        alt={title}
        className="mx-auto max-h-40 w-70 rounded-lg border border-gray-300"
      />

      <form onSubmit={handleSubmit(onSubmit)} className="flex flex-col gap-4">
        {inputs.map((input) => (
          <InputField
            key={input.name}
            type={input.type}
            placeholder={input.placeholder}
            labelName={input.labelName}

```

```

    required={input.required}
    {...register(input.name, { valueAsNumber: input.valueAsNumber })}
  />
))}
<input type="submit" className="hidden" ref={submitRef} />
</form>

<div className="flex h-full items-end justify-end gap-x-2">
  <button
    type="button"
    className="rounded-md bg-neutral-100 px-3 py-1.5 font-medium hover:bg-neutral-200
active:bg-neutral-300 dark:border dark:border-stone-800 dark:bg-neutral-900 dark:text-
gray-200 dark:hover:bg-stone-800"
    onClick={() => openFilePicker()}
  >
    Ввести з файлу
  </button>
  <button
    className="rounded-md bg-blue-500 px-3 py-1.5 font-medium text-white hover:bg-
blue-700 dark:bg-gray-200 dark:text-black dark:hover:bg-neutral-400"
    type="submit"
    onClick={() => submitRef.current?.click()}
  >
    Обрахувати
  </button>
</div>
</main>
)
}

export type { Input }
export default GenericPage

```

input-field.tsx

```

import { forwardRef } from 'react'

interface InputFieldProps {
  name: string
  type: 'string' | 'number'
  placeholder: string
  labelName: string
}

const InputField = forwardRef<HTMLInputElement, InputFieldProps>(
  ({ name, type, placeholder, labelName, ...rest }, ref) => {
    return (
      <div className="mx-auto flex items-center gap-x-4">
        <label htmlFor={name} className="text-gray-600 dark:text-zinc-400">
          {labelName}
        </label>
        <input
          type={type}

```

```

      id={name}
      name={name}
      className="rounded-lg border border-gray-300 p-2 px-3 hover:border-gray-400
focus:border-blue-500 focus:outline-none dark:border-stone-700 dark:bg-neutral-950
dark:text-neutral-400 dark:hover:bg-stone-900 dark:focus:border-white"
      placeholder={placeholder}
      ref={ref}
      {...rest}
    />
  </div>
)
}
)

```

```

InputField.displayName = 'InputField'

```

```

export default InputField

```

is-json.tsx

```

const isValidJson = (str: string): boolean => {
  try {
    JSON.parse(str)
  } catch (e) {
    return false
  }
  return true
}

```

```

export default isValidJson

```

show-dialog.tsx

```

const showDialog = (type: 'info' | 'error', title: string, message: string) => {
  window.api.showDialog({ type, title, message })
}
export default showDialog

```

```

contextBridge.exposeInMainWorld('api', {
  showDialog(details: Electron.MessageBoxOptions) {
    ipcRenderer.invoke('show-dialog', details)
  }
})

```

```

ipcMain.handle('show-dialog', async (event, details) => {
  dialog.showMessageBox({
    type: details.type,
    buttons: ['OK'],
    title: details.title,
    detail: details.detail,
    message: details.message
  })
})

```

Скриншоти, що підтверджують виконання роботи:

Vite + React + TS

Алгоритми

Лінійні

Розгалужувальні

Циклічні

Крадожон Максим
Романович
ІО-32 • 15 варіант

Лінійний алгоритм

$$Y1 = \left(a + \frac{b}{x}\right)^3 + \left(b + \frac{a}{x}\right)^5$$

Значення А:

Значення В:

Значення Х:

Ввести з файлу

Vite + React + TS

Алгоритми

Лінійні

Розгалужувальні

Циклічні

Крадожон Максим
Романович
ІО-32 • 15 варіант

Лінійний алгоритм

$$Y1 = \left(a + \frac{b}{x}\right)^3 + \left(b + \frac{a}{x}\right)^5$$

Значення А:

Значення В:

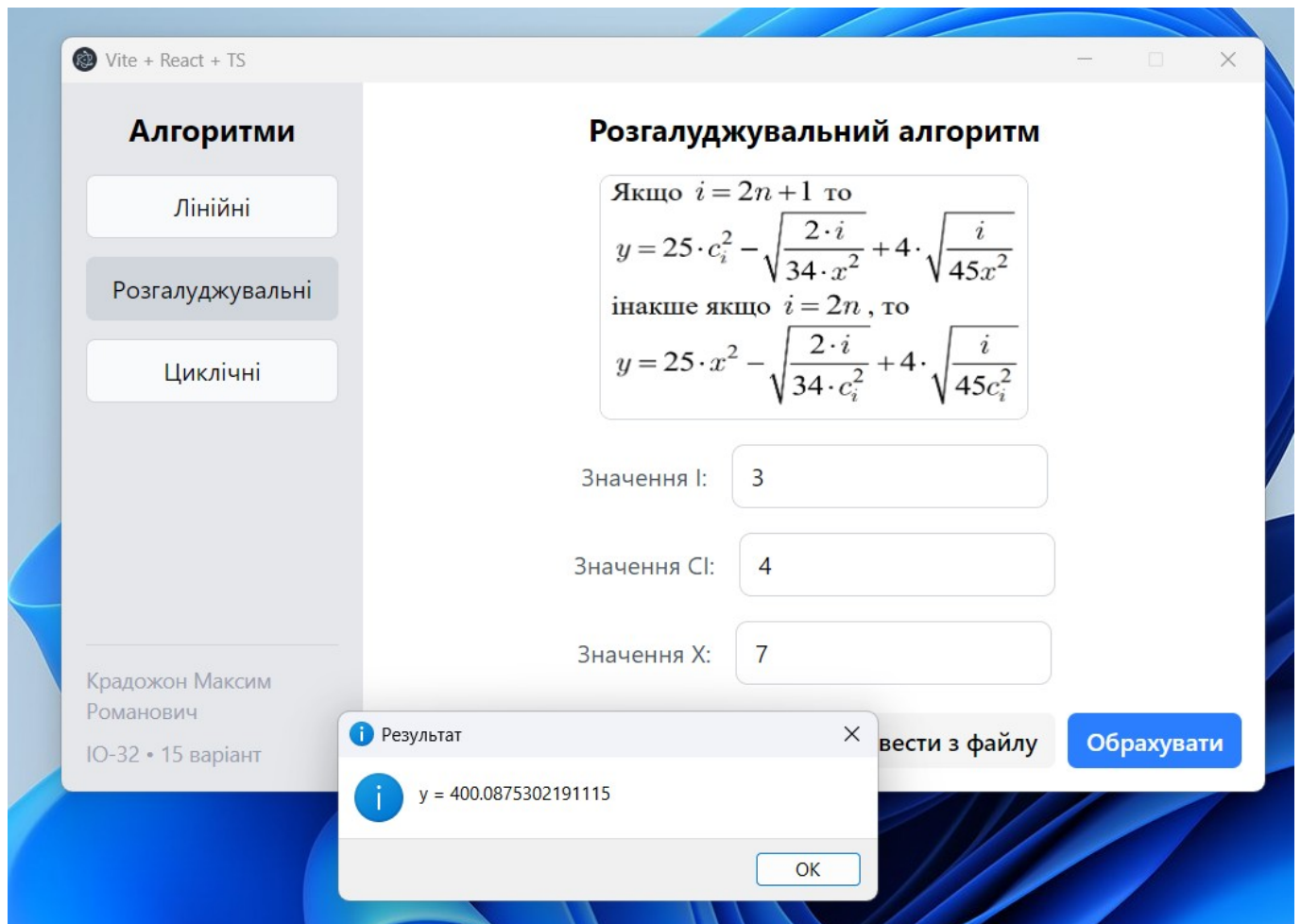
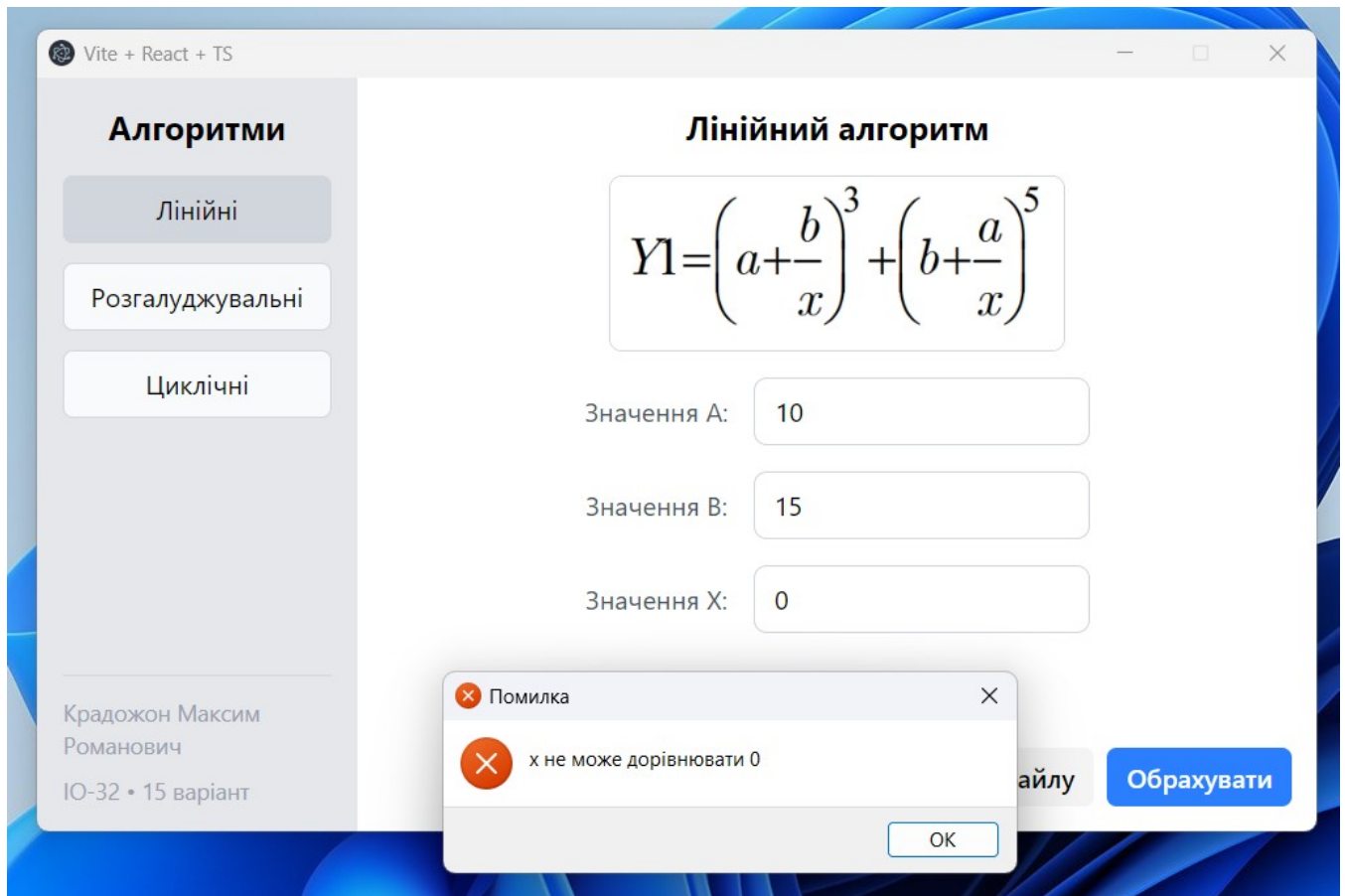
Значення Х:

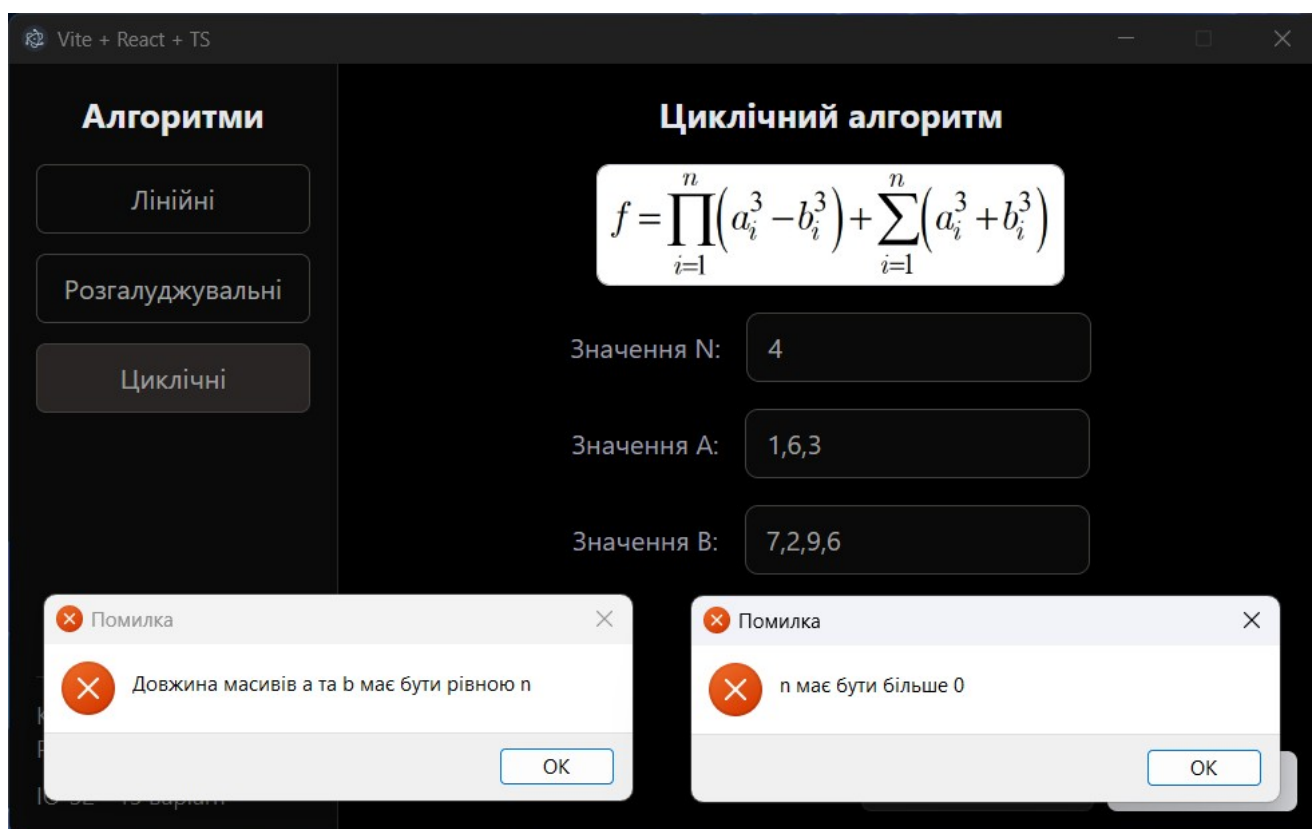
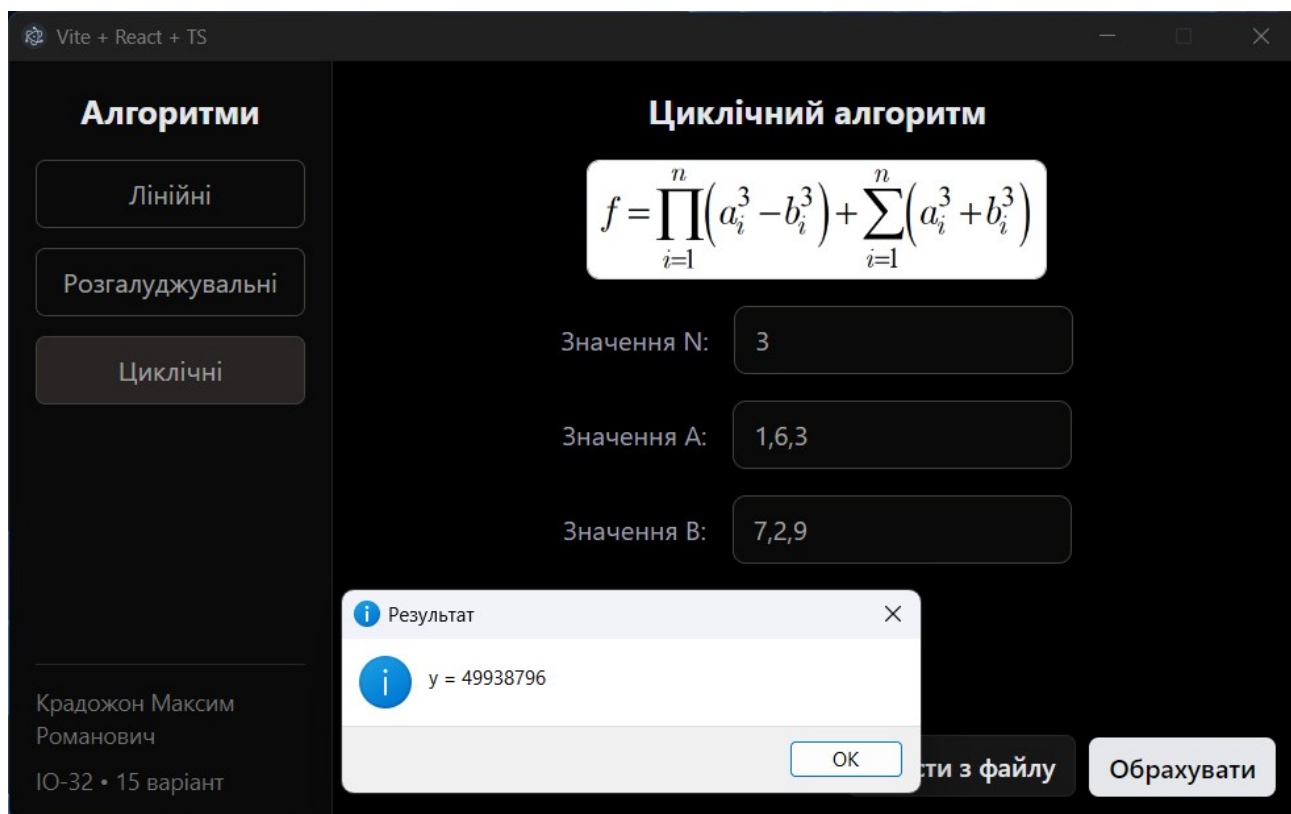
Ввести з файлу

Результат

$y = 3205359.375$

OK





Висновок: Під час виконання лабораторної роботи, я написав програму на мові програмування JavaScript для обчислення трьох алгоритмів: лінійного, того що розгалужується та циклічного. Створив блок-схеми для кожного з алгоритмів. Забезпечив введення даних з клавіатури або документу та обробку всіх можливих помилок.